深度学习在光伏功率预测中的应用研究

雷显秋

（中国矿业大学，江苏省徐州市铜山区大学路 1 号 221116 ）

Application of Deep Learning in Photovoltaic Power Forecasting

Xianqiu Lei

（China University of Mining and Technology, No. 1 Daxue Road, Tongshan District, Xuzhou City, 221116, China）

ABSTRACT: As renewable energy penetration increases, power systems are becoming more variable and uncertain, making generation forecasting a fundamental capability in smart energy systems. This report analyzes the role of deep learning in smart energy, focusing on photovoltaic power forecasting while also discussing applications in wind power forecasting and microgrid energy management. Representative models, engineering value, and current challenges are summarized, including data quality, cross-scenario generalization, interpretability, and deployment cost. An experiment on the public LIGHT photovoltaic dataset further compares Persistence, MLP, and LSTM for 90-minute-ahead forecasting after resampling 1-minute power data to 45-minute intervals and adding temporal features. The results show that LSTM performs best on MAE, RMSE, and MAPE, achieving an RMSE of 0.3630, about 13.28% lower than Persistence. This indicates that under longer forecasting horizons and coarser time granularity, LSTM can better capture temporal dependencies in photovoltaic power series. The experimental code is available at `https://github.com/glance02/SmartEnergyPVForecasting`.

KEY WORDS: smart energy systems; deep learning; photovoltaic power forecasting; wind power forecasting; microgrid energy management

摘要：随着风电、光伏等新能源装机持续增长，电力系统运行的不确定性显著增强，出力预测已成为智慧能源中的基础能力。本文围绕深度学习在智慧能源中的应用展开分析，重点讨论光伏功率预测，并结合风电预测与微电网能量管理，概述其代表性模型、工程价值及面临的数据质量、跨场景泛化、可解释性和部署成本等问题。基于公开 LIGHT 光伏数据集，本文进一步比较了 Persistence、MLP 和 LSTM 在 90 分钟 ahead 预测任务中的表现：将 1 分钟功率数据重采样为 45 分钟数据，并加入时间周期特征。实验结果表明，LSTM 在 MAE、RMSE 和 MAPE 三项指标上均优于 Persistence 和 MLP，其中 RMSE 为 0.3630，较 Persistence 下降约 13.28%，说明在更长预测步长和更粗时间粒度下，LSTM 对光伏功率时序依赖的建模能力更强。实验代码开源在 GitHub 仓库：`https://github.com/glance02/SmartEnergyPVForecasting`。

关键词：智慧能源；深度学习；光伏功率预测；风电预测；微电网能量管理

1 引言

智慧能源的核心并不是简单地把能源系统“数字化”，而是通过感知、通信、计算和控制技术，让发电、输配、储能和用能环节实现更高水平的协同。随着分布式光伏、电动汽车、储能设备和柔性负荷大量接入，能源系统表现出更强的时变性和复杂性。传统依赖经验规则或线性假设的方法在很多场景下已经难以满足精度与实时性要求，尤其是在新能源渗透率不断提升之后，风电和光伏出力的不确定性会明显放大调度难度。

在这一背景下，预测成为智慧能源中的基础能力。无论是电力负荷预测、光伏功率预测还是微电网能量管理，背后都需要对历史时序、气象条件和设备状态进行建模。深度学习的优势在于可以自动提取多层次特征，更适合处理高维、强非线性和带有长时依赖的数据，因此逐渐成为智慧能源研究中的重要技术路线。

不过，深度学习在智慧能源中的意义并不只是“把误差做得更小”。从工程角度看，一个真正有价值的模型还应该具备足够的鲁棒性、解释性和部署可行性，能够在天气突变、传感器缺失和场景迁移时保持相对稳定的表现，并最终服务于调度与控制决策。因此，本文不只介绍模型名称，而是围绕“深度学习如何提升智慧能源应用效果”这一主线，对光伏预测这一典型问题展开分析，并进一步延伸到风电预测和微电网优化。

本文后续内容安排如下：第二部分介绍智慧能源中深度学习的典型任务、常见模型与评价指标；第三部分结合代表性文献分析光伏预测、风电预测

和微电网能量管理中的应用案例；第四部分基于公开光伏数据集开展可复现实验，对比 Persistence、MLP 和 LSTM 模型的预测效果；最后总结深度学习在智慧能源应用中的价值、局限与未来发展方向。

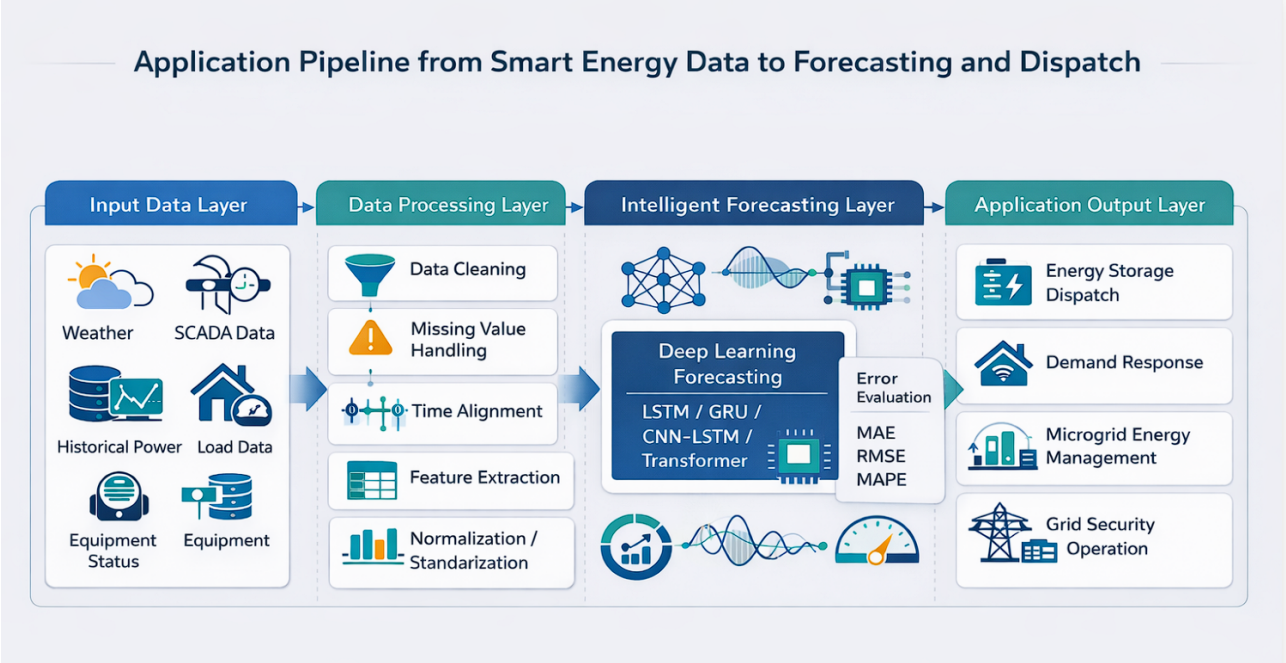

图 1 智慧能源数据到预测与调度的应用链路图

Fig. 1 Application chain from data to forecasting and dispatch in smart energy

2 智慧能源中的深度学习应用基础

2.1 典型任务

从应用角度看，智慧能源中的深度学习任务可以概括为三类：

1） 预测类任务，如电力负荷预测、风电功率预测、光伏功率预测和建筑能耗预测

2） 识别与诊断类任务，如故障识别、异常检测和设备健康评估

3） 决策与控制类任务，如微电网能量管理、储能调度和需求响应优化

以上三类任务虽然形式不同，但都高度依赖数据质量、时间关联和场景适应性，因此非常适合使用数据驱动模型建模。

在本文聚焦的新能源预测场景中，深度学习的优势主要体现在三个方面。第一，新能源出力受辐照度、温度、云层变化、风速和风向等因素共同影响，呈现明显非线性，传统线性模型很难稳定描述这种关系。第二，功率序列本身存在较强时间依赖，LSTM 和 GRU 等循环结构更适合捕捉这种动态变化。第三，随着采集条件改善，模型输入已经不再局限于历史功率一个变量，而是可以同时包含多种气象与设备特征，多源数据融合成为进一步提升精度的重要方向。

2.2 常见模型与评价指标

目前智慧能源研究中最常见的深度学习模型包括 MLP、LSTM、GRU、CNN、TCN、Transformer 及其混合结构。MLP 作为基础前馈神经网络，适合对固定长度特征向量进行非线性映射，常作为时序预测中的对比基线。LSTM 和 GRU 对长期依赖关系建模能力较强，因此在负荷与新能源预测中应用最广。CNN 可以提取局部模式，常与 LSTM 结合形成 CNN-LSTM，用于同时学习局部变化与时间依赖。TCN 通过膨胀卷积增强了长序列建模能力，在高频多步预测中表现突出。近年来，Transformer 和注意力机制也逐步进入新能源预测领域，用于处理更长时间跨度和更复杂的多变量关系。

预测模型通常采用 MAE、RMSE、MAPE 等指标进行评价。这三个指标的计算方式如下：

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

$$\mathrm{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

其中，y_i 表示真实值，$\hat{y}_i$ 表示预测值，n 表示样本数。

MAE 便于理解平均偏差大小，RMSE 对大误差更敏感，更适合衡量极端天气或波动时段下的稳定性，MAPE 则便于跨场景比较，但在真实值接近零时容易失真，因此在夜间光伏出力场景下需要谨慎解释。对智慧能源而言，模型价值不能只看平均误差，还应结合鲁棒性和下游调度收益综合判断。

表 1 智慧能源典型任务、输入数据与常用模型

Table 1 Typical tasks, inputs, and models in smart energy

任务	常见输入数据	常用模型
电力负荷预测	历史负荷、气温、节假日、时段标签	LSTM、GRU、Transformer、XGBoost
光伏功率预测	历史功率、辐照度、温度、湿度、云量	LSTM、BiLSTM、CNN-LSTM、TCN
风电功率预测	SCADA、风速、风向、气压、数值天气预报	LSTM、TCN、GNN、Attention
微电网能量管理	负荷、发电、储能状态、电价、运行约束	DQN、DDPG、MPC+ 深度学习

3 代表性研究与案例分析

深度学习方法在光伏预测领域已经成为主流路线。Mellit 等人在光伏输出功率预测综述中指出，光伏预测已经从传统统计方法逐步转向机器学习与深度学习，并且随着多源气象数据与更高频采样数据的可获得性提升，深度学习的重要性持续增强 [7]。该文强调，光伏预测并不是单纯的时间序列回归问题，而是与天气、季节、地理位置和时间尺度紧密相关的综合建模任务。这个结论说明，模型选择不能脱离具体应用背景。

Khouili 等人在 2025 年的系统综述中进一步指出，在纳入分析的 26 篇深度学习光伏预测研究中，LSTM 是使用频率最高的架构，占比 32.69%，CNN 紧随其后，占比 28.85% [3]。同时，多源数据融合、可解释性和鲁棒性仍然是未来研究重点。由此可以看出，深度学习已经成为光伏预测的主流技术路线，但工程化问题依然没有完全解决。

Mellit、Massi Pavan 和 Lughi 基于意大利 Trieste 大学微电网的真实光伏数据，对 LSTM、BiLSTM、GRU、BiGRU、CNN1D、CNN-LSTM 和 CNN-GRU 等网络进行了系统比较 [1]。该研究同时覆盖 1 分钟、5 分钟、30 分钟和 60 分钟等多种时间尺度，因此非常适合说明不同模型在实际光伏场景中的性能差异。研究表明，在一步预测且时间尺度较短时，结构相对简单的 LSTM 或 GRU 已经可以取得较高精度；而当任务转为多步预测、天气更复杂时，模型性能差异会被明显放大。

该案例说明深度学习在短时光伏预测中确实具有优势，但预测步长越长，问题就越困难。它的意义不仅在于证明“某个模型效果更好”，更在于提供了统一数据源、多模型、多时间尺度比较的研究框架，这比只展示单次最优结果更有说服力。

Luo、Zhang 和 Zhu 提出的 PC-LSTM 为光伏预测提供了很有启发性的思路 [2]。该研究尝试把领域知识和物理约束引入深度学习框架，以提升小时级和日前光伏功率预测的合理性与稳定性。作者指出，在训练数据稀疏或场景变化明显时，纯数据驱动方法容易得到“误差看起来不高、但在物理上并不合理”的结果，而加入领域知识的模型往往表现更稳健。

这一思路对智慧能源具有普遍意义。真实系统中的很多问题都不是“数据越多、网络越深”就一定更好。气象异常、传感器故障和站点迁移都会导致纯黑箱模型性能下降。因此，把物理约束、设备边界或经验规律融入模型，是深度学习在能源场景中真正走向可用的重要方向。

Sarmas 等人在 2023 年提出了一种基于元学习的短时光伏预测方法，用多个 LSTM 变体作为基础模型，再通过元学习器动态组合预测结果 [8]。该方法在葡萄牙里斯本的三套屋顶光伏系统上进行了验证，并且不依赖数值天气预报输入。研究结果显示，该方法相比单一最佳基础模型可带来最高约 5% 的精度提升，相比简单平均集成也有进一步改善。

该案例的重要性在于，它说明深度学习研究的重点正在从“谁的单模型更强”逐渐转向“如何提升跨场景泛化与稳定性”。对智慧能源这类强场景依赖问题来说，泛化能力往往比单站点上的最低误差更重要。

尽管本文主线聚焦光伏预测，但风电预测同样能反映深度学习在智慧能源中的发展趋势。Liu 和 Zhang 在 2024 年的综述中系统梳理了 140 余篇深度学习风电预测研究，指出风电预测正在从传统历史功率或单点气象输入，逐渐过渡到结合 SCADA、数值天气预报和图结构信息的更复杂建模框架 [4]。这说明新能源预测正在从单序列建模走向时空协同建模。

更具体地说，Meka、Alaeddini 和 Bhaganagar 基于一个 130 MW、包含 86 台风机的真实风场数据，利用 TCN 模型对 0 至 50 分钟的短时风电功率进行多步预测，并与 LSTM、CNN-LSTM 和多元线性回归模型进行比较 [5]。结果表明，TCN 在不同风速区间下都表现出更稳定的优势。这个案例可以作为光伏研究的横向补充，说明深度学习在“高频、波动、强时序依赖”的新能源任务上具有普遍适用性。

若只讨论预测误差，容易忽略智慧能源的最终目标是更优运行。Alabdullah 和 Abido 在微电网能量管理研究中采用 Deep Q-Network 构建强化学习

调度框架，并在真实微电网案例上验证了方法有效性 [6]。研究表明，该方法在随机负荷、分布式电源和电价波动条件下能够获得接近最优调度的结果，其运行成本与最优调度方案相比相差在 2% 以内。

这一研究提醒我们，预测模型的真正价值不在于单独优化某个误差指标，而在于为储能充放电、购电决策、负荷转移和系统安全留出更好的决策空间。“预测”与“调度”是相互联系的关系，而不是两个孤立的任务。

表 2 代表性论文案例对比

Table 2 Comparison of representative studies

文献	场景与数据	模型/方法	关键发现
Mellit 等，2021 [1]	Trieste 大学微电网光伏数据；1、5、30、60 分钟尺度	LSTM、BiLSTM、GRU、CNN-LSTM 等	一步预测精度高，说明深度学习适合短时光伏预测；但多步预测仍具挑战，且预测步长越长越难。
Luo 等，2021 [2]	小时级、日前光伏功率预测	PC-LSTM	融合领域知识后模型更稳健，在稀疏数据下更有优势，说明机理与数据融合优于纯黑箱建模。
Khouili 等，2025 [3]	26 篇深度学习光伏预测研究综述	系统综述	LSTM 与 CNN 最常见，鲁棒性与解释性仍是挑战，说明深度学习已成主流，但工程化问题仍突出。
Liu 和 Zhang，2024 [4]	140 余篇风电预测研究综述	系统综述	风电预测正从时序模型走向时空与图结构建模，说明深度学习在新能源预测中具有跨场景适用性。
Meka 等，2021 [5]	130 MW 风场、86 台风机、12 个月数据	TCN vs. LSTM/CNN-LSTM/MLR	TCN 在多步短时预测中更稳定，说明高频多步预测需要更强的时序建模能力。
Alabdullah 和 Abido，2022 [6]	真实微电网能量管理案例	DQN 强化学习	运行成本与最优调度相差在 2% 以内，说明预测最终应服务于调度优化。
Sarmas 等，2023 [8]	葡萄牙三套屋顶光伏系统	元学习 + 多 LSTM	相比单一最佳基础模型可提升约 5%，说明泛化能力与动态集成是重要方向。

4 光伏功率预测实验

4.1 实验目的

为了避免结课报告完全停留在文献综述层面，我配套设计了一个可在本地复现的实验，用于验证深度学习在光伏功率短时预测中的基本效果。实验目标不是追求发表级精度，而是通过统一的数据流程和对比框架，直观展示深度学习模型与简单基线模型之间的差异，从而增强报告的实践性和可信度。实验代码开源在我的 GitHub 仓库，如图 2：`https://github.com/glance02/SmartEnergyPVForecasting`。

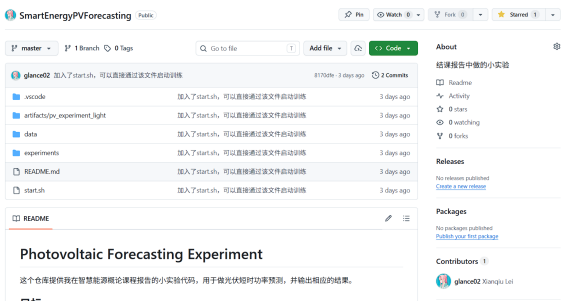

图 2 开源代码界面

Fig. 2 Source code interface

该实验采用 Zenodo 平台公开数据集“LIGHT dataset sample for solar smart home pilot - 201801”中的一个真实光伏站点样本，选取 `ID00002` 作为研究对象。原始数据为 1 分钟粒度的功率序列，相邻时刻之间连续性很强，因此实验将其重采样为 45 分钟粒度，以更清晰地呈现光伏功率在昼夜转换、白天爬升、峰值波动和回落过程中的变化特征，并保留单站点功率序列作为核心输入。配套实验脚本位于 `experiments/`中，`experiments/main.py` 用于完成训练、评估并保存模型权重和实验配置，`experiments/vis.py` 基于这些结果生成预测曲线图和简短报告摘要。核心实现见附录 A，完整代码与 README 文件可在 GitHub 仓库中查看。

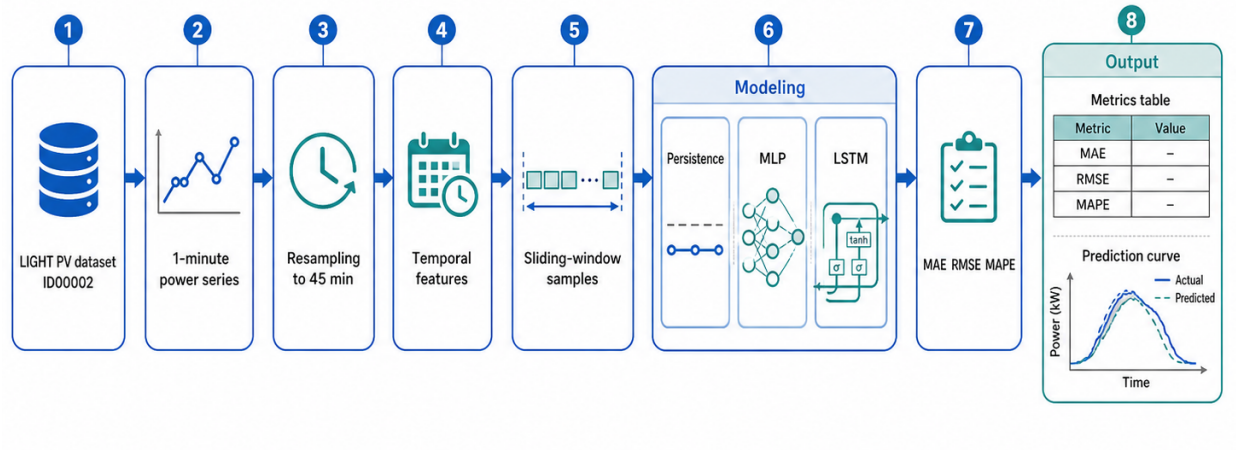

图 3 光伏功率预测实验流程

Fig. 3 Workflow of the photovoltaic power forecasting experiment

图 3 展示了本文光伏功率预测实验的整体流程。实验首先从 LIGHT 数据集中选取 `ID00002` 站点的 1 分钟功率序列，然后依次进行 45 分钟重采样、时间周期特征构造和滑动窗口样本生成。在建模阶段，本文将 Persistence、MLP 和 LSTM 放在同一数据划分与评价口径下比较，最后通过 MAE、RMSE 和 MAPE 形成误差指标表，并结合预测曲线观察不同模型对光伏功率变化趋势的刻画能力。该流程说明本文实验不是单纯展示某个模型结果，而是围绕统一的数据处理、模型对比和结果评价链路展开。

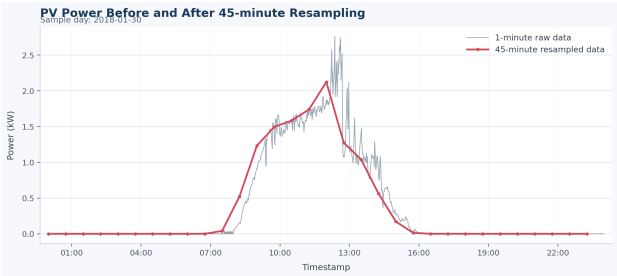

图 4 原始 1 分钟数据与 45 分钟重采样数据对比

Fig. 4 Comparison between 1-minute raw data and 45-minute resampled data

由图 4 可以看出，45 分钟重采样保留了光伏功率日内爬升、峰值和回落的主要趋势，同时削弱了 1 分钟数据中的局部高频波动，使后续模型比较更聚焦于较长时间尺度下的时序依赖关系。

4.2 实验方案

实验设置三类模型。第一类是 Persistence 基线，即“未来时刻功率约等于当前时刻功率”，这是时序预测中非常常见且很有参考价值的强基线。第二类是 MLP 模型，它把历史窗口展开后输入前馈神经网络，用来表示“没有显式时序记忆结构的神经网络”能达到什么水平。第三类是单层 LSTM 模型，用来体现深度学习对序列依赖关系的建模优势。LSTM 也是老师在课程中介绍过、我在课堂展示中也进一步讲解过的经典时序模型。

本实验采用以下配置。首先，将原始 1 分钟数据重采样为 45 分钟数据；其次，加入基于时间戳构造的周期性时间特征，包括昼夜周期和周内周期；再次，使用过去 12 个时间步，也就是过去 9 小时的数据，预测未来 2 个时间步，也就是未来 90 分钟的光伏功率。数据经过时间排序后，保留完整的昼夜功率变化，不使用夜间过滤，并按 70%/15%/15% 切分为训练集、验证集和测试集。训练阶段采用验证集 RMSE 做早停，并对重采样粒度、预测步长、窗口长度、隐藏层宽度和 batch size 做了简单调参，最终选择 `hidden_size=128`、`batch_size=32` 这一组参数。这样的设置既保留了全天连续功率曲线，也使数据同时呈现夜间低值、白天爬升、峰值波动和回落等不同阶段的特征，从而更贴近真实光伏功率序列在较长时间尺度下的时序结构。

4.3 实验模型介绍

我在这个实验中主要对比了三类模型：Persistence、MLP 和 LSTM。以下简要介绍每一个模型。

1. Persistence：这是一个非常简单的基线模型，直接将当前时刻的功率值作为未来时刻的预测值。虽然它没有任何学习能力，但在某些短时预测场景下，尤其是当功率变化较为平稳时，Persistence 模型可能会表现得相当不错，因此是一个重要的参考点。

$$\hat{y}_{t+h} = y_t, \quad h = 1, 2, \ldots, H$$

其中，y_t 表示当前时刻的真实功率值，$\hat{y}_{t+h}$ 表示对未来第 h 个时间步的预测值，H 表示预测步数。在本文实验设置中，$H = 2$。

2. MLP：多层感知机（Multilayer Perceptron）是一种前馈神经网络，它将历史窗口内的功率值和时间特征展开成一个固定长度的输入向量，经过若干全连接层后输出未来时刻的功率预测。MLP 没有显式的时序建模能力，因此它的表现可以反映出在没有考虑时间依赖关系的情况下，单纯基于历史数据进行预测的效果。

MLP 的前向传播过程可以表示为：

$$\mathbf{h}^{(l)} = f^{(l)}\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right)$$

$$\hat{y} = \mathbf{W}^{(L+1)}\mathbf{h}^{(L)} + \mathbf{b}^{(L+1)}$$

其中，$\mathbf{x}$ 表示输入特征向量，$\mathbf{h}^{(l)}$ 表示第 l 层的隐藏表示，$\mathbf{h}^{(0)} = \mathbf{x}$ 表示网络以输入向量作为初始输入，$\mathbf{W}^{(l)}$ 和 $\mathbf{b}^{(l)}$ 分别表示第 l 层的权重矩阵和偏置向量，$f^{(l)}(\cdot)$ 表示激活函数，$l = 1, 2, \ldots, L$ 表示隐藏层编号，L 表示隐藏层总数，$\hat{y}$ 表示模型输出。

3. LSTM：长短期记忆网络（Long Short-Term Memory）是一种特殊的循环神经网络，设计用来捕捉序列数据中的长期依赖关系。如图 5，LSTM 通过引入门控机制，能够有效地保留和利用历史信息，从而更好地建模光伏功率序列中存在的时间依赖性和非线性变化。这使得 LSTM 在时序预测任务中通常能够显著优于简单的基线模型和没有时序结构的神经网络。

LSTM 的核心在于通过输入门、遗忘门和输出门控制信息的写入、保留与输出，其基本计算过程可以表示为：

$$\begin{aligned}
\mathbf{i}_t &= \sigma(\mathbf{W}_{xi}\mathbf{x}_t + \mathbf{W}_{hi}\mathbf{h}_{t-1} + \mathbf{b}_i),\\
\mathbf{f}_t &= \sigma(\mathbf{W}_{xf}\mathbf{x}_t + \mathbf{W}_{hf}\mathbf{h}_{t-1} + \mathbf{b}_f),\\
\tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_{xc}\mathbf{x}_t + \mathbf{W}_{hc}\mathbf{h}_{t-1} + \mathbf{b}_c),\\
\mathbf{o}_t &= \sigma(\mathbf{W}_{xo}\mathbf{x}_t + \mathbf{W}_{ho}\mathbf{h}_{t-1} + \mathbf{b}_o)
\end{aligned}$$

$$\begin{aligned}
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t,\\
\mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t),\\
\hat{y}_{t+H} &= \mathbf{W}_y\mathbf{h}_t + b_y
\end{aligned}$$

其中，$\mathbf{i}_t$、$\mathbf{f}_t$ 和 $\mathbf{o}_t$ 分别表示输入门、遗忘门和输出门，$\tilde{\mathbf{c}}_t$ 表示候选记忆状态，$\mathbf{c}_t$ 和 $\mathbf{h}_t$ 分别表示当前时刻的单元状态和隐藏状态，$\odot$ 表示逐元素乘法。在本文实验中，模型利用历史窗口最后一个时刻的隐藏状态输出未来第 H 个时间步的功率预测值。

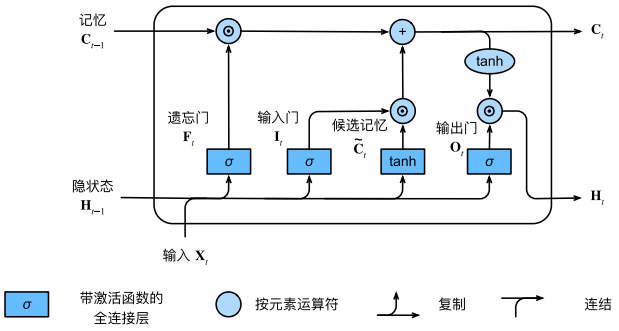

图 5 LSTM 原理解析

Fig. 5 LSTM principle illustration

其实目前在智慧能源领域，除了 LSTM 以外，还有很多其他类型的深度学习模型也被广泛应用，例如 GRU、CNN-LSTM、TCN 和 Transformer 等，尤其是 Transformer。但是在这个实验中，我选择了 LSTM 作为代表性的深度学习模型，主要是因为它在时序预测领域的经典地位和广泛应用，同时也为了保持实验的简洁性和可复现性。未来如果有机会，我也计划在这个基础上进一步引入其他类型的模型进行对比分析，以更全面地展示深度学习在光伏功率预测中的潜力和适用性。

4.4 实验结果

基于上述真实数据和配置，我最终保留的最佳实验结果如表 3 所示。从表 3 可以看出，在“45 分钟重采样 + 时间特征 + 90 分钟 ahead 预测 + 保留完整昼夜序列”的设置下，LSTM 在 MAE、RMSE 和 MAPE 三项指标上都表现最好。其中，LSTM 的 RMSE 为 0.3630，较 Persistence 下降约 13.28%，较 MLP 下降约 17.95%。这说明即便不使用夜间过滤，只要适当拉长时间粒度和预测步长，LSTM 对较长时序依赖和非线性变化的建模优势依然能够体现出来。

表 3 实验结果

Table 3 Experimental results on real data

Model	MAE	RMSE	MAPE(%)
LSTM	0.1951	0.3630	56.49
Persistence	0.2306	0.4186	139.77
MLP	0.2538	0.4424	64.61

需要说明的是，Persistence 的 MAPE 数值依然偏大，主要是因为光伏功率在夜间、日出和日落等低功率时段接近零，百分比误差容易被放大。因此，在解释结果时应更关注 RMSE 和 MAE 这类对工程应用更稳定的指标。综合来看，该实验支持这样一个判断：在完整昼夜预测设定下，深度学习模型能否优于强基线，并不只取决于是否保留夜间样本，还与重采样粒度、预测步长和历史窗口长度等

实验口径密切相关。当前这组 45 分钟重采样、90 分钟 ahead 的设置下，LSTM 在保持完整时间轴的前提下取得最优结果。

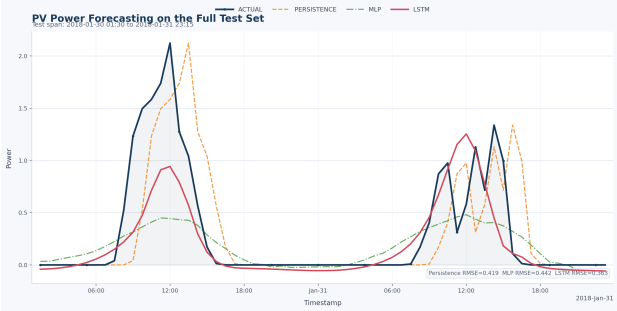

图 6 测试集整段时间上的真实值与各模型预测曲线

Fig. 6 Actual and predicted curves on the test set

4.5 意义与启示

本实验的意义在于，它在真实光伏数据上给出了一个结构清晰、过程完整且结果可解释的验证案例。通过与 Persistence 和 MLP 的对比，可以更具体地说明深度学习在光伏功率预测中的优势主要体现在对时序依赖关系和非线性变化的建模上，而这种优势并不是无条件成立的，仍然受到数据质量、特征设计、时间粒度和预测步长等因素的共同影响。因而，这一实验不仅补充了前文的文献分析，也进一步说明了深度学习在智慧能源场景中的有效性与适用边界。

需要指出的是，本文实验仍属于小规模验证实验，主要用于说明深度学习模型在光伏功率预测中的基本效果。实验只选取了单一站点和一个月份的数据，输入特征也以历史功率和时间周期特征为主，尚未引入完整的数值天气预报、云量、辐照度等外部气象信息。因此，实验结果不能直接代表所有光伏场景下的模型性能。后续若扩大到多站点、多季节和多气象特征条件下进行验证，结论会更具有工程推广价值。

5 结论

尽管深度学习在智慧能源中展现出明显潜力，但其工程化应用仍面临若干关键问题。首先是数据问题。智慧能源系统中的数据往往来自不同设备、不同采样频率和不同协议，存在缺失值、异常值、噪声和时间对齐误差。如果数据预处理不到位，再复杂的模型也难以稳定发挥作用。尤其在光伏预测中，云量突变、局地遮挡和传感器故障都会显著影响模型泛化。

其次是跨场景泛化问题。很多论文在单一站点上可以取得很好的结果，但换到另一地区、另一设备或另一季节后，误差会明显增大。这说明模型可能学到的是“站点特征”，而不是更一般的能源规律。元学习、迁移学习和跨站点集成是解决这一问题的重要方向，但目前仍处于快速发展阶段。

再次是可解释性问题。能源系统属于典型的高可靠性场景，调度人员通常不会仅凭“模型误差较小”就完全信任一个黑箱系统。未来如果要将深度学习更大规模地用于调度与控制，就必须给出更明确的解释机制，例如特征贡献分析、置信区间预测、物理边界校验和异常检测联动等。

最后是预测与决策之间的脱节问题。许多研究把预测误差作为唯一目标，但真实系统更关心的是调度成本、备用容量、安全约束和用户侧体验。未来智慧能源中的高价值方向，很可能不是孤立地追求更小的 RMSE，而是把预测、优化和控制纳入统一框架中，实现“预测-决策-反馈”闭环。

深度学习正在成为智慧能源研究中的关键支撑技术，尤其在光伏和风电等新能源预测任务中表现出明显优势。通过对代表性文献的分析可以看出，LSTM、GRU、CNN-LSTM、TCN 等模型能够较好地处理强时序依赖与复杂非线性关系，在短时预测场景中具有较高应用价值。同时，领域知识融合、元学习和强化学习调度案例表明，深度学习的价值并不止于提高单一预测指标，而是能够进一步服务于储能控制、微电网运行和能源管理优化。

本文配套实验也验证了这一点：在公开 LIGHT 光伏数据集上，LSTM 在 90 分钟提前预测任务中取得了最低 RMSE，优于 Persistence 和 MLP。这说明在适当的时间粒度和预测步长下，具备序列记忆能力的深度学习模型能够更好地刻画光伏功率的时间依赖关系。

但也应看到，深度学习并不是智慧能源问题的“万能解”。如果数据质量不足、模型缺乏约束、结果难以解释，那么即使测试集误差较小，也未必能在真实系统中稳定落地。因此，本文最终的判断是：深度学习在智慧能源中的核心意义，不是把模型做得更复杂，而是利用数据驱动方法提升系统对不确定性的感知、预测和应对能力。未来真正高价值的方向，将是可解释、可迁移、可部署，并能直接支撑能源决策的深度学习方法。

参考文献

[1] MELLIT A, MASSI PAVAN A, LUGHI V. Deep learning neural networks for short-term photovoltaic power forecasting[J]. Renewable Energy, 2021, 172: 276-288. DOI: 10.1016/j.renene.2021.02.166.

[2] LUO X, ZHANG D, ZHU X. Deep learning based forecasting of photovoltaic power generation by incorporating domain knowledge[J]. Energy, 2021, 225: 120240. DOI: 10.1016/j.energy.2021.120240.

[3] KHOUILI O, HANINE M, LOUZAZNI M, et al. Evaluating the impact of deep learning approaches on solar and photovoltaic power forecasting: A systematic re-

view[J]. Energy Strategy Reviews, 2025, 59: 101735. DOI: 10.1016/j.esr.2025.101735.

[4] LIU H, ZHANG Z. Development and trending of deep learning methods for wind power predictions[J]. Artificial Intelligence Review, 2024, 57: 112. DOI: 10.1007/s10462-024-10728-z.

[5] MEKA R, ALAEDDINI A, BHAGANAGAR K. A robust deep learning framework for short-term wind power forecast of a full-scale wind farm using atmospheric variables[J]. Energy, 2021, 221: 119759. DOI: 10.1016/j.energy.2021.119759.

[6] ALABDULLAH M H, ABIDO M A. Microgrid energy management using deep Q-network reinforcement learning[J]. Alexandria Engineering Journal, 2022, 61(11): 9069-9078. DOI: 10.1016/j.aej.2022.02.042.

[7] MELLIT A, MASSI PAVAN A, OGLIARI E, et al. Advanced methods for photovoltaic output power forecasting: A review[J]. Applied Sciences, 2020, 10(2): 487. DOI: 10.3390/app10020487.

[8] SARMAS E, SPILIOTIS E, STAMATOPOULOS E, et al. Short-term photovoltaic power forecasting using meta-learning and numerical weather prediction independent Long Short-Term Memory models[J]. Renewable Energy, 2023, 216: 118997. DOI: 10.1016/j.renene.2023.118997.

A 实验核心代码

本附录列出光伏功率预测实验中的核心代码片段，主要包括模型定义、数据预处理、时间特征构造、滑动窗口样本生成、训练评估流程和结果保存逻辑。完整实验代码见 GitHub 仓库：`https://github.com/glance02/SmartEnergyPVForecasting`。

A.1 模型定义

```python
class MLPRegressor(nn.Module):
    def __init__(self, input_size: int, hidden_size: int) -> None:
        super().__init__()
        mid_size = max(hidden_size // 2, 16)
        self.network = nn.Sequential(
            nn.Linear(input_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, mid_size),
            nn.ReLU(),
            nn.Linear(mid_size, 1),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        flattened = x.reshape(x.size(0), -1)
        return self.network(flattened).squeeze(-1)


class LSTMRegressor(nn.Module):
    def __init__(self, num_features: int, hidden_size: int) -> None:
        super().__init__()
        self.lstm = nn.LSTM(
            input_size=num_features,
            hidden_size=hidden_size,
            num_layers=1,
            batch_first=True,
        )
        self.output = nn.Linear(hidden_size, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        encoded, _ = self.lstm(x)
        return self.output(encoded[:, -1, :]).squeeze(-1)
```

Listing 1 MLP 与 LSTM 模型定义

A.2 数据预处理与时间特征

```python
def prepare_frame(
    frame: pd.DataFrame,
    time_col: str,
    target_col: str,
    extra_features: list[str],
) -> pd.DataFrame:
    required = [time_col, target_col, *extra_features]
    missing = [column for column in required if column not in frame.columns]
    if missing:
        raise KeyError(f"Missing required columns: {missing}")

    prepared = frame.copy()
    prepared[time_col] = pd.to_datetime(prepared[time_col], errors="coerce")
    prepared = prepared.dropna(subset=[time_col]).drop_duplicates(subset=[time_col], keep="last")
    prepared = prepared.sort_values(time_col).reset_index(drop=True)

    for column in [target_col, *extra_features]:
        prepared[column] = pd.to_numeric(prepared[column], errors="coerce")

    prepared = prepared.dropna(subset=[target_col, *extra_features]).reset_index(drop=True)
    if prepared.empty:
        raise ValueError("No valid rows remain after preprocessing.")
    return prepared


def resample_frame(
    frame: pd.DataFrame,
    time_col: str,
    value_cols: list[str],
    rule: str,
) -> pd.DataFrame:
    if not rule:
```

```python
        return frame.copy()

    resampled = (
        frame.set_index(time_col)[value_cols]
        .resample(rule)
        .mean()
        .dropna()
        .reset_index()
    )
    if resampled.empty:
        raise ValueError(f"Resampling with rule '{rule}' removed all rows.")
    return resampled


def add_time_features(frame: pd.DataFrame, time_col: str) -> tuple[pd.DataFrame, list[str]]:
    enriched = frame.copy()
    timestamp = pd.to_datetime(enriched[time_col])
    minute_of_day = timestamp.dt.hour * 60 + timestamp.dt.minute
    day_of_week = timestamp.dt.dayofweek

    enriched["time_of_day_sin"] = np.sin(2 * np.pi * minute_of_day / (24 * 60)).astype(np.float32)
    enriched["time_of_day_cos"] = np.cos(2 * np.pi * minute_of_day / (24 * 60)).astype(np.float32)
    enriched["day_of_week_sin"] = np.sin(2 * np.pi * day_of_week / 7).astype(np.float32)
    enriched["day_of_week_cos"] = np.cos(2 * np.pi * day_of_week / 7).astype(np.float32)

    return enriched, [
        "time_of_day_sin",
        "time_of_day_cos",
        "day_of_week_sin",
        "day_of_week_cos",
    ]
```

Listing 2 数据清洗、重采样与周期性时间特征构造

A.3 滑动窗口样本构造

```python
def create_sequences(
    frame: pd.DataFrame,
    feature_cols: list[str],
    target_col: str,
    time_col: str,
    window_size: int,
    horizon: int,
    feature_scaler: Standardizer,
    target_scaler: Standardizer,
) -> dict[str, np.ndarray]:
    raw_features = frame[feature_cols].to_numpy(dtype=np.float32)
    raw_target = frame[target_col].to_numpy(dtype=np.float32)
    timestamp_series = pd.to_datetime(frame[time_col]).reset_index(drop=True)
    timestamps = timestamp_series.astype("string").to_numpy()

    scaled_features = feature_scaler.transform(raw_features.astype(np.float32)).astype(np.float32)
    scaled_target = target_scaler.transform(raw_target.reshape(-1, 1)).astype(np.float32).reshape(-1)

    x_scaled, x_raw, y_scaled, y_raw, ts = [], [], [], [], []

    deltas = timestamp_series.diff().dropna()
    positive_deltas = deltas[deltas > pd.Timedelta(0)]
    if positive_deltas.empty:
        segment_ids = pd.Series(np.zeros(len(frame), dtype=np.int32))
    else:
        expected_delta = positive_deltas.mode().iloc[0]
        gap_threshold = expected_delta * 1.5
        segment_breaks = timestamp_series.diff() > gap_threshold
        segment_ids = segment_breaks.fillna(False).cumsum()

    for _, segment_index in segment_ids.groupby(segment_ids).groups.items():
        segment_index = np.asarray(segment_index, dtype=np.int64)
        if len(segment_index) < window_size + horizon:
            continue

        for end in range(window_size, len(segment_index) - horizon + 1):
            start = end - window_size
            target_index = end + horizon - 1

            window_indices = segment_index[start:end]
            target_row = segment_index[target_index]
```

```python

            x_scaled.append(scaled_features[window_indices])
            x_raw.append(raw_features[window_indices])
            y_scaled.append(scaled_target[target_row])
            y_raw.append(raw_target[target_row])
            ts.append(timestamps[target_row])

    return {
        "x_scaled": np.asarray(x_scaled, dtype=np.float32),
        "x_raw": np.asarray(x_raw, dtype=np.float32),
        "y_scaled": np.asarray(y_scaled, dtype=np.float32),
        "y_raw": np.asarray(y_raw, dtype=np.float32),
        "timestamp": np.asarray(ts, dtype=object),
    }
```

Listing 3 基于历史窗口和预测步长构造监督学习样本

A.4 指标计算与模型训练

```python
def compute_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict[str, float]:
    y_true = np.asarray(y_true, dtype=np.float64)
    y_pred = np.asarray(y_pred, dtype=np.float64)
    mae = float(np.mean(np.abs(y_true - y_pred)))
    rmse = float(np.sqrt(np.mean(np.square(y_true - y_pred))))
    threshold = max(float(np.max(np.abs(y_true))) * 0.05, 1e-3)
    valid = np.abs(y_true) >= threshold
    if not np.any(valid):
        valid = np.abs(y_true) >= 1e-3
    denominator = np.clip(np.abs(y_true[valid]), threshold, None)
    mape = float(np.mean(np.abs((y_true[valid] - y_pred[valid]) / denominator)) * 100.0)
    return {"mae": mae, "rmse": rmse, "mape": mape}


def predict_scaled(model: nn.Module, loader: DataLoader, device: torch.device) -> tuple[np.ndarray, np.ndarray]:
    model.eval()
    preds, targets = [], []
    with torch.inference_mode():
        for batch_x, batch_y in loader:
            batch_x = batch_x.to(device)
            pred = model(batch_x).cpu().numpy()
            preds.append(pred)
            targets.append(batch_y.numpy())
    return np.concatenate(preds), np.concatenate(targets)


def train_model(
    model: nn.Module,
    train_loader: DataLoader,
    val_loader: DataLoader,
    target_scaler: Standardizer,
    epochs: int,
    device: torch.device,
) -> nn.Module:
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    criterion = nn.MSELoss()
    patience = 8
    best_rmse = math.inf
    best_state: dict[str, torch.Tensor] | None = None
    stale_epochs = 0

    model.to(device)

    for epoch in range(1, epochs + 1):
        model.train()
        running_loss = 0.0
        sample_count = 0
        for batch_x, batch_y in train_loader:
            batch_x = batch_x.to(device)
            batch_y = batch_y.to(device)

            optimizer.zero_grad()
            pred = model(batch_x)
            loss = criterion(pred, batch_y)
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * batch_x.size(0)
```

```python
            sample_count += batch_x.size(0)

        val_pred_scaled, val_true_scaled = predict_scaled(model, val_loader, device)
        val_pred = inverse_target(target_scaler, val_pred_scaled)
        val_true = inverse_target(target_scaler, val_true_scaled)
        val_rmse = compute_metrics(val_true, val_pred)["rmse"]
        train_loss = running_loss / max(sample_count, 1)
        print(
            f"Epoch {epoch:03d} | train_loss={train_loss:.5f} | "
            f"val_rmse={val_rmse:.5f}"
        )

        if val_rmse + 1e-6 < best_rmse:
            best_rmse = val_rmse
            best_state = copy.deepcopy(model.state_dict())
            stale_epochs = 0
        else:
            stale_epochs += 1
            if stale_epochs >= patience:
                print(f"Early stopping at epoch {epoch}.")
                break

    if best_state is None:
        raise RuntimeError("Training did not produce a valid checkpoint.")

    model.load_state_dict(best_state)
    return model
```

Listing 4 误差指标、预测函数与早停训练过程